

Design Document: Vector Database Engine

1. System Overview

The Vector Database Engine consists of three separated components: the network layer, the vector store, and the IVF index. The network layer accepts incoming TCP connections and parses client commands. The vector store holds all vector data and IDs in memory using a flat array and a hash map for lookups. The IVF index is built on top of the vector store to enable fast approximate nearest neighbor search via kmeans clustering. When a client sends a SEARCH command using the IVF index, the request flows through all three components. First, the network layer receives the request and parses the query vector, the desired number of neighbors, and the search parameters. Second, the request goes to the IVF index to identify the closest centroids and select the vectors to scan. Finally, the search module retrieves these vectors from the vector store, computes the squared Euclidean distances, and returns the sorted nearest neighbors to the network layer, which sends the results back to the client.

2. Network Layer

The server achieves concurrency by spawning a separate thread for each client to handle incoming connections. The communication protocol is line based, meaning each client command is sent as a single line terminated by a newline character. The parser reads the first token of the line to identify the command type and then processes the remaining arguments. If the command has an invalid number of arguments or is unrecognized, the server responds with an error message. The QUIT command terminates the client session and closes the socket while the server continues running to accept other clients. When a client triggers the BUILD command, it blocks all other client operations because the building process holds the global mutex for its entire duration.

3. Vector Store and Brute Force Search

Data Layout

The vector database stores all vector components contiguously in a single flat array of 32 bit float values. For a database configured with dimension D , the vector at internal index i occupies D consecutive float positions starting at index $i * D$. A parallel array of 64 bit integer IDs stores the external identifier for each vector, where the ID at index i matches the vector at index i . To handle fast overwrite lookups when adding vectors, a custom hash map maps each external ID to its corresponding internal index. This flat memory layout is chosen because it is highly cache friendly, ensures sequential memory access during scans, and makes serialization straightforward.

Distance Function

The engine uses squared Euclidean distance as the similarity metric, which computes the sum of the squared differences of components across all D dimensions. Calculating the square root is skipped during distance computations because the square root is a monotonic function and omitting it does not change the relative ranking of the nearest neighbors. This optimization avoids expensive operations inside the tight inner loop that iterates over the D dimensions.

Brute Force kNN

For brute force search, the engine scans every stored vector in the flat store. It maintains a max heap of size k to track the k smallest distances encountered during the scan. If a vector has a smaller distance than the maximum distance in the heap, the heap pops its largest element and inserts the new vector. After completing the full scan of the store, the engine pops all elements from the heap, sorts them in ascending order of distance, and returns them. The final response to the client includes a summary line reporting the exact number of vectors scanned.

4. IVF Index

kmeans Clustering

The BUILD command runs kmeans clustering using Lloyd's algorithm to organize vectors into clusters. The number of clusters K is set to the floor of the square root of the total number of vectors N , which is a standard rule of thumb. For example, if there are 50000 vectors, the database creates 223 clusters. The algorithm initializes centroids by randomly selecting K vectors from the store without replacement. During each iteration, the engine assigns each vector to its nearest centroid and then recomputes each centroid as the mean of all vectors assigned to that cluster. The process repeats until convergence, which occurs when no vector changes its cluster assignment, or when the execution reaches a hard limit of 50 iterations. If any cluster becomes empty during clustering, it is left in place without being removed or reseeded.

Index Layout

The IVF index layout consists of a flat array representing the K centroids, where each centroid contains D float components. The index also contains a list for each cluster that stores the internal indices of the vectors belonging to it. The vectors themselves remain in the main flat store, and the IVF index only holds these integer references to avoid duplicating vector data in memory.

IVF Search

IVF search proceeds in four main steps. First, the engine computes the distance from the query vector to all K centroids. Second, it selects the n_{probe} closest centroids by maintaining a max heap of size n_{probe} . Third, for each of the selected centroids, the engine scans only the vectors belonging to those clusters, updating a top k max heap with the closest vectors. Finally, the engine extracts the results from the heap, sorts them by distance, and returns them to the client along with the count of scanned vectors. The expected query speedup over brute force search is approximately K divided by n_{probe} .

Incremental Insertion After BUILD

When a new vector is added after the IVF index is built, the engine finds the nearest centroid by scanning all K centroids. It then appends the new vector's internal index to that cluster's list. The centroids are not recomputed during this insertion, which causes the index quality to degrade slightly over time as more vectors are added. Rerunning the BUILD command reclusters all vectors from scratch to restore search quality.

5. Concurrency

The engine ensures thread safety by using a single global mutex defined within the vector store class. Every ADD command locks this mutex for the entire duration of the insertion to prevent race conditions during array resizing or hash map updates. Similarly, every SEARCH command locks the mutex during its scanning phase to ensure it reads a consistent state of the vectors. The BUILD command holds the global mutex for its entire execution time, which blocks all other clients from inserting or searching until clustering completes. This simple lock design provides correctness and is suitable for the data sizes targeted by the project.

6. Snapshot File Format

The database persists its state to a binary snapshot file. The file begins with a 4 byte magic header spelling VDB1. This header is followed by a 32 bit unsigned integer version field which is set to 1, a 32 bit unsigned integer for the vector dimension, a 32 bit unsigned integer for the vector count N , and a 32 bit unsigned integer for the cluster count K which is set to 0 if the IVF index has not been built. Following this header, the file contains N 64 bit integer ID values, followed by the $N * D$ 32 bit float vector components stored contiguously. If K is greater than 0, the $K * D$ 32 bit float centroid values are written next. Finally, for each of the K clusters, a 32 bit unsigned integer specifies the size of the cluster, followed by that many 32 bit unsigned integers representing the internal indices of the vectors belonging to that cluster. All integers are stored in little endian format, and float values use the standard IEEE 754 single precision binary32 representation. To prevent file corruption in the event of a system crash, the database writes the data to a temporary file named snapshot.vdb.tmp and then renames it to snapshot.vdb. This rename operation is atomic on Linux systems. The SAVE command returns a success response showing the number of saved vectors, while the LOAD command returns a success message confirming the number of loaded vectors. The database does not load the snapshot automatically at startup, so the LOAD command must be called manually after a server restart.

7. Benchmark Results

Here are the actual benchmark results from our run:

Mode	nprobe	Avg Query Time (ms)	Recall@10	Avg Vectors Scanned
BRUTE	-	66.463	1.0000	50000
IVF	1	18.221	0.0590	230
IVF	5	47.521	0.1920	1145
IVF	10	47.518	0.3120	2285
IVF	25	51.591	0.5160	5725

At nprobe=1, the IVF index is the fastest with a query time of 18.221 ms, but the recall is at its lowest at 0.0590. As we increase the nprobe value, the recall gets better but the query time also gets slower. For example, at nprobe=25, the query time increases to 51.591 ms and the recall improves to 0.5160. Theoretically, recall should exceed 0.9 at nprobe=10 while remaining faster than brute force, but here the recall is only 0.3120 at 47.518 ms compared to 66.463 ms for brute force. This

speed difference happens because of the number of vectors we scan. Brute force scans all 50000 vectors, while IVF at nprobe=1 only scans 230 vectors, and nprobe=25 scans 5725 vectors.

8. Bonus: Cosine Similarity

Euclidean Distance Benchmarks

Mode	nprobe	Avg Query Time (ms)	Recall@10	Avg Vectors Scanned
BRUTE	-	66.463	1.0000	50000
IVF	1	18.221	0.0590	230
IVF	5	47.521	0.1920	1145
IVF	10	47.518	0.3120	2285
IVF	25	51.591	0.5160	5725

Cosine Similarity Benchmarks

Mode	nprobe	Avg Query Time (ms)	Recall@10	Avg Vectors Scanned
BRUTE	-	61.736	1.0000	50000
IVF	1	47.749	0.0740	224
IVF	5	47.625	0.2290	1123
IVF	10	47.585	0.3510	2245
IVF	25	50.093	0.5550	5607

Cosine similarity measures the angle between vectors instead of their absolute distance. When we insert vectors, we normalize them to unit length, which means cosine distance reduces to one minus the dot product and runs just as fast as Euclidean distance. For example, brute force search takes 61.736 ms for Cosine compared to 66.463 ms for Euclidean. Looking at the tables, the IVF recall values are very similar for both metrics, like 0.0740 for Cosine and 0.0590 for Euclidean at nprobe=1, and 0.5550 for Cosine and 0.5160 for Euclidean at nprobe=25. They behave similarly because random normal vectors in high dimensions tend to be almost orthogonal, so the angular differences correlate closely with Euclidean distances. Normalization also handles the zero vector edge case by preventing division by zero when calculating angles.